PRÁCTICA 3: DASH

Mónica Ortega Menéndez
Jorge Velasco de María
Andrés Roques Sánchez
Adrián Modino Merino

Índice

Parte 1: Introducción a dash.js 5
Parte 2: Generación de videos para streaming y visualización básica de métricas 8
Parte 3: Gestión DRM con dash.js 17
Parte 4: Visualización avanzada de métricas 20

Índice de figuras

Ilustración 1. Documento MPD 5
Ilustración 2. Visualizado del contenido 7
Ilustración 3. Comando de generación del video a calidad baja 8
Ilustración 4. Comando de generación del video a calidad media 8
Ilustración 5. Comando de generación del video a calidad alta 9
Ilustración 6. Comando de codificación en MPEG del video de calidad baja 9
Ilustración 7. Comando de codificación en MPEG del video de calidad media 9
Ilustración 8. Comando de codificación en MPEG del video de calidad alta 9
Ilustración 9. Comandos de segmentación de video 10
Ilustración 10. Comandos de preparación DASH de video 10
Ilustración 11. Documento MPD de la segmentación 10
Ilustración 12. Limitaciones de red del navegador. 12
Ilustración 13. Visualización del video a alta calidad. 13
Ilustración 14. Visualización del video a calidad media. 13
Ilustración 15. Visualización del video a baja calidad. 14
Ilustración 16. Visualización de mensajes GET - 200 OK dinámicos del servidor. 14
Ilustración 17. Visualización de mensajes de redirección debido a las cachés habilitadas. 15
Ilustración 18. Mensajes del servidor web en la consola 15
Ilustración 19. Encriptación de videos fragmentados 17
Ilustración 20. Comportamiento MPEG-DASH con DRM 17
Ilustración 21. Visualización de contenido encriptado 18
Ilustración 22. Mensajes de error de la consola por video sin acceso 18
Ilustración 23. Métrica de streaming del video a calidad alta 20
Ilustración 24. Métrica de streaming del cambio de video a calidad alta a media 21
Ilustración 25. Métrica de streaming del video a calidad media 21
Ilustración 26. Métrica de streaming del cambio de video a calidad media a baja 22
Ilustración 27. Métrica de streaming del cambio de video a calidad media a baja (llenado de buffer) 22
Ilustración 28. Métrica de streaming del video a calidad baja 23

Índice de tablas

Tabla 1. Configuraciones 8

Parte 1: Introducción a dash.js

El objetivo de esta primera parte de la práctica es implementar un cliente web básico utilizando la librería dash.js que permita visualizar un vídeo adaptativo en streaming conforme al estándar MPEG-DASH. Se utilizará un video codificado con diferentes calidades para analizar el funcionamiento del protocolo DASH.

En el fichero index.html, ubicado a la misma altura que dash.min.all.js, se define el cliente web. En el contenido del fichero se define la URL del archivo MPD, que es un documento XML que describe la estructura del contenido adaptativo.

El contenido del documento MPD es el siguiente:



Ilustración 1. Documento MPD

Entre los campos más importantes del documento MPD se encuentran:

XML: El MPD está estructurado como un documento XML, que permite representar datos jerárquicos. Se abre con la cabecera: <?xml version="1.0" encoding="UTF-8"?>

MPD: Define las propiedades globales del contenido y el protocolo DASH. Entre las propiedades definidas destacan:

- type="static", lo que indica que no es una retransmisión en vivo.
- minBufferTime="PT5.000S", el tiempo mínimo que debe estar en el buffer antes de iniciar la reproducción es de 5 segundos.
- maxSegmentDuration="PT2.005S", la duración máxima de cada segmento es de 2.005 segundos.
- mediaPresentationDuration="PT193.680S" indica que la duración total del contenido es de 193,68 segundos.

Period: Define un segmento temporal del contenido. En este caso solo existe un periodo, de id period0, que abarca toda la duración del video.

AdaptationSet: Define un grupo de diferentes representaciones, es decir diferentes calidades, agrupadas por tipo de medio.

En este caso tenemos dos AdaptationSets uno para el video (mimeType= "video/mp4") y otro para el audio (mimeType="audio/mp4")

En el caso del set del video también se define maxWidth="1920" maxHeight="1080" y maxFrameRate="30000/1001" lo que indica que la representación de mayor calidad dentro de este conjunto alcanza una resolución Full HD (1920x1080) con el indicado framerate máximo.

SegmentTemplate: Define la forma en que se construyen las URLs para los segmentos. Incluye:

- initialization="$RepresentationID$-Header.m4s", archivo de iniciación necesario para cada representación.
- media="$RepresentationID$-270146-i-$Number$.m4s", indica los archivos de los segmentos.
- duration="179704", indica la duración de cada segmento con relación al timescale (90000). Cada segmento durará 1,9967 segundos.

Representation: Dentro de cada AdaptationSet, hay varias representaciones que corresponden a distintas calidades. El cliente puede cambiar entre estas representaciones en función del ancho de banda disponible.

Cada representation especifica un bandwidth, codec, width, height, framerate y scanType

A continuación se incluye una captura de pantalla del reproductor en ejecución, tras lanzar el servidor web dentro del contenedor Docker con Python (Ilustración 2).

El reproductor dash.js selecciona automáticamente la mejor calidad posible según el ancho de banda disponible. A medida que se cambia la velocidad de conexión, el reproductor adapta la calidad en tiempo real sin interrumpir la reproducción.

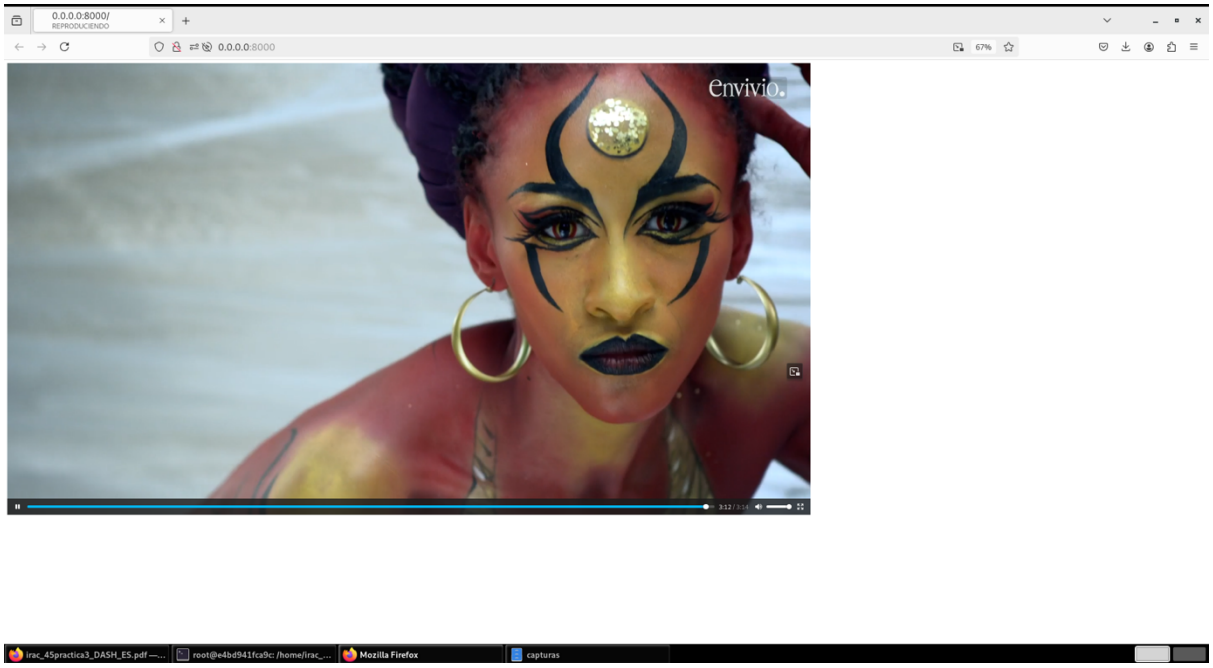


Ilustración 2. Visualizado del contenido

Parte 2: Generación de videos para streaming y visualización básica de métricas

Esta parte de la práctica se enfoca en la generación de diferentes configuraciones del video (resolución y bitrate) para ser ofrecido en streaming a través de MPEG-DASH.

Generación de diferentes configuraciones de video

Se crearán las siguientes tres configuraciones:

Calidad	Nombre	Bitrate teórico	Bitrate real	Resolución	Path
Baja	low_confid.267	100 kb/s	96,57 kb/s	160x90	/home/irac_p3/
Media	mid_confid.267	600 kb/s	579,42 kb/s	640x360	/home/irac_p3/
Alta	high_confid.267	2400 kb/s	2317,48 kb/s	1280x720	/home/irac_p3/

Tabla 1. Configuraciones

Para generar cada una de estas configuraciones se han utilizado los siguientes comandos, donde se emplea el comando de ejecución de x264, partiendo de la base del archivo video_p3.mp4 contenido en el Docker.

```
root@f427f5ae91fb:/home/irac_p3# ./x264 --output low_config.264 --fps 24 --preset slow --bitrate 100 --vbv-maxrate 4800 --vbv-bufsize 9600 --min-keyint 48 --keyint 48 --scenecut 0 --no-scenecut --pass 1 --video-
filter "resize:width=160,height=90" video_p3.mp4
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x3ad74f80] stream 0, timescale not set
lavf [info]: 1920x1080p 1:1 @ 25/1 fps (cfr)
resize [info]: resizing to 160x90
x264 [info]: using SAR=1/1
x264 [info]: using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
x264 [info]: profile Main, level 3.0, 4:2:0, 8-bit
x264 [info]: frame I:193   Avg QP:27.56  size:  1960
x264 [info]: frame P:3908  Avg QP:30.95  size:   681
x264 [info]: frame B:5128  Avg QP:32.66  size:   313
x264 [info]: consecutive B-frames: 21.2% 11.7%  7.3% 59.7%
x264 [info]: mb I  I16..4:  8.8%  0.0% 91.2%
x264 [info]: mb P  I16..4: 18.5%  0.0%  0.0%  P16..4: 76.2%  0.0%  0.0%  0.0%  0.0%    skip: 5.4%
x264 [info]: mb B  I16..4:  7.2%  0.0%  0.0%  B16..8: 35.0%  0.0%  0.0%  direct: 9.8%  skip:48.0%  L0:23.1% L1:27.5% BI:49.4%
x264 [info]: final ratefactor: 26.53
x264 [info]: direct mvs  spatial:35.3% temporal:64.7%
x264 [info]: coded y,uvDC,uvAC intra: 86.7% 53.9% 22.6% inter: 35.6% 12.5% 0.7%
x264 [info]: i16 v,h,dc,p: 15% 18% 46% 22%
x264 [info]: i4 v,h,dc,ddl,ddr,vr,hd,vl,hu: 12% 20% 16%  9%  9%  6% 11%  6% 11%
x264 [info]: i8c dc,h,v,p: 65% 18% 13%  4%
x264 [info]: Weighted P-Frames: Y:20.0% UV:4.0%
x264 [info]: kb/s:96.57

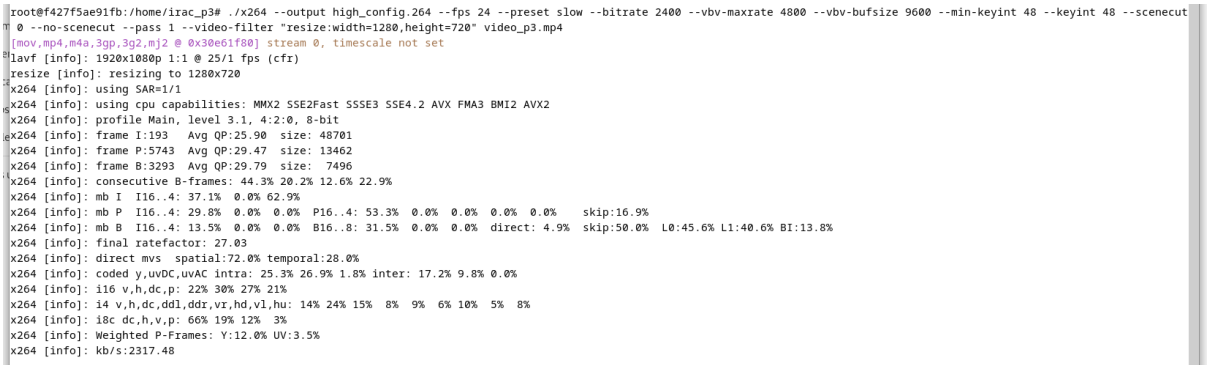
encoded 9229 frames, 231.24 fps, 96.57 kb/s
```

Ilustración 3. Comando de generación del video a calidad baja

```
root@f427f5ae91fb:/home/irac_p3# ./x264 --output mid_config.264 --fps 24 --preset slow --bitrate 600 --vbv-maxrate 4800 --vbv-bufsize 9600 --min-keyint 48 --keyint 48 --scenecut 0 --no-scenecut --pass 1 --video-
filter "resize:width=640,height=360" video_p3.mp4
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x1233ff80] stream 0, timescale not set
lavf [info]: 1920x1080p 1:1 @ 25/1 fps (cfr)
resize [info]: resizing to 640x360
x264 [info]: using SAR=1/1
x264 [info]: using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
x264 [info]: profile Main, level 3.0, 4:2:0, 8-bit
x264 [info]: frame I:193   Avg QP:28.99  size: 13951
x264 [info]: frame P:4563  Avg QP:32.58  size:  3616
x264 [info]: frame B:4473  Avg QP:33.42  size:  1935
x264 [info]: consecutive B-frames: 30.0% 12.1% 12.3% 45.6%
x264 [info]: mb I  I16..4: 35.0%  0.0% 65.0%
x264 [info]: mb P  I16..4: 25.4%  0.0%  0.0%  P16..4: 57.4%  0.0%  0.0%  0.0%  0.0%    skip:17.2%
x264 [info]: mb B  I16..4: 12.1%  0.0%  0.0%  B16..8: 32.5%  0.0%  0.0%  direct: 5.0%  skip:50.5%  L0:42.0% L1:43.2% BI:14.8%
x264 [info]: final ratefactor: 29.27
x264 [info]: direct mvs  spatial:63.3% temporal:36.7%
x264 [info]: coded y,uvDC,uvAC intra: 34.4% 29.4% 3.9% inter: 15.0% 8.1% 0.0%
x264 [info]: i16 v,h,dc,p: 21% 27% 31% 21%
x264 [info]: i4 v,h,dc,ddl,ddr,vr,hd,vl,hu: 13% 22% 17%  8% 10%  6% 11%  5%  9%
x264 [info]: i8c dc,h,v,p: 71% 16% 10%  2%
x264 [info]: Weighted P-Frames: Y:16.0% UV:4.4%
x264 [info]: kb/s:579.42

encoded 9229 frames, 208.88 fps, 579.42 kb/s
```

Ilustración 4. Comando de generación del video a calidad media



```
root@f427f5ae91fb:/home/irac_p3# ./x264 --output high_config.264 --fps 24 --preset slow --bitrate 2400 --vbv-maxrate 4800 --vbv-bufsize 9600 --min-keyint 48 --keyint 48 --scenecut
 0 --no-scenecut --pass 1 --video-filter "resize:width=1280,height=720" video_p3.mp4
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x30e61f80] stream 0, timescale not set
lavf [info]: 1920x1080p 1:1 @ 25/1 fps (cfr)
resize [info]: resizing to 1280x720
x264 [info]: using SAR=1/1
x264 [info]: using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
x264 [info]: profile Main, level 3.1, 4:2:0, 8-bit
x264 [info]: frame I:193   Avg QP:25.90  size: 48701
x264 [info]: frame P:5743  Avg QP:29.47  size: 13462
x264 [info]: frame B:3293  Avg QP:29.79  size:  7496
x264 [info]: consecutive B-frames: 44.3% 20.2% 12.6% 22.9%
x264 [info]: mb I  I16..4: 37.1%  0.0% 62.9%
x264 [info]: mb P  I16..4: 29.8%  0.0%  0.0%  P16..4: 53.3%  0.0%  0.0%  0.0%  0.0%    skip:16.9%
x264 [info]: mb B  I16..4: 13.5%  0.0%  0.0%  B16..8: 31.5%  0.0%  0.0%  direct: 4.9%  skip:50.0%  L0:45.6% L1:40.6% BI:13.8%
x264 [info]: final ratefactor: 27.03
x264 [info]: direct mvs  spatial:72.0% temporal:28.0%
x264 [info]: coded y,uvDC,uvAC intra: 25.3% 26.9% 1.8% inter: 17.2% 9.8% 0.0%
x264 [info]: i16 v,h,dc,p: 22% 30% 27% 21%
x264 [info]: i4 v,h,dc,ddl,ddr,vr,hd,vl,hu: 14% 24% 15%  8%  9%  6% 10%  5%  8%
x264 [info]: i8c dc,h,v,p: 66% 19% 12%  3%
x264 [info]: Weighted P-Frames: Y:12.0% UV:3.5%
x264 [info]: kb/s:2317.48
```

Ilustración 5. Comando de generación del video a calidad alta

Posteriormente estas 3 configuraciones deben ser codificadas en MPEG. Para ello se utilizaron los siguientes comandos de MP4BOX, donde se generan los archivos de extensión .mp4.

```
root@f427f5ae91fb:/home/irac_p3# ./gpac_public/bin/gcc/MP4Box -add low_config.264 -fps 24 low_config.mp4
Track Importing MPEG-4 AVC - Width 160 Height 90 FPS 24/1 SAR 1/1
AVC|H264 Import results: 9229 samples (9617 NALUs) - Slices: 194 I 3908 P 5128 B - 1 SEI - 193 IDR - 0 CRA
AVC|H264 Stream uses forward prediction - stream CTS offset: 3 frames
AVC|H264 Max NALU size is 3247 - stream could be optimized by setting nal_length=2
0.500 secs Interleaving
```

Ilustración 6. Comando de codificación en MPEG del video de calidad baja

```
root@f427f5ae91fb:/home/irac_p3# ./gpac_public/bin/gcc/MP4Box -add mid_config.264 -fps 24 mid_config.mp4
Track Importing MPEG-4 AVC - Width 640 Height 360 FPS 24/1 SAR 1/1
AVC|H264 Import results: 9229 samples (9617 NALUs) - Slices: 194 I 4563 P 4473 B - 1 SEI - 193 IDR - 0 CRA
AVC|H264 Stream uses forward prediction - stream CTS offset: 3 frames
AVC|H264 Max NALU size is 25405 - stream could be optimized by setting nal_length=2
0.500 secs Interleaving
```

Ilustración 7. Comando de codificación en MPEG del video de calidad media

```
root@f427f5ae91fb:/home/irac_p3# ./gpac_public/bin/gcc/MP4Box -add high_config.264 -fps 24 high_config.mp4
Track Importing MPEG-4 AVC - Width 1280 Height 720 FPS 24/1 SAR 1/1
AVC|H264 Import results: 9229 samples (9617 NALUs) - Slices: 194 I 5743 P 3293 B - 1 SEI - 193 IDR - 0 CRA
AVC|H264 Stream uses forward prediction - stream CTS offset: 3 frames
0.500 secs Interleaving
```

Ilustración 8. Comando de codificación en MPEG del video de calidad alta

Una vez generados correctamente los tres archivos .mp4 con sus respectivos bitrates, utilizaremos el framework Bento4, el cual ya se encuentra instalado en el contenedor Docker.

Tras revisar el directorio Bento4/bin, donde se encuentran las distintas herramientas que ofrece este framework, emplearemos la herramienta de segmentación de vídeo

para procesar los tres vídeos generados anteriormente, utilizando los comandos mostrados en la Ilustración 9.

```
root@c45f5528b694:/home/irac_p3# ./Bento4/bin/mp4fragment low_config.mp4 low_config_f.mp4
found regular I-frame interval: 48 frames (at 24.000 frames per second)
root@c45f5528b694:/home/irac_p3# ./Bento4/bin/mp4fragment mid_config.mp4 mid_config_f.mp4
found regular I-frame interval: 48 frames (at 24.000 frames per second)
root@c45f5528b694:/home/irac_p3# ./Bento4/bin/mp4fragment high_config.mp4 high_config_f.mp4
found regular I-frame interval: 48 frames (at 24.000 frames per second)
root@c45f5528b694:/home/irac_p3#
```

Ilustración 9. Comandos de segmentación de video

Posteriormente a tener los videos fragmentados, con el siguiente comando mp4dash se prepararán los videos para su presentación mediante DASH:

```
root@c45f5528b694:/home/irac_p3# ./Bento4/bin/mp4dash high_config_f.mp4 mid_config_f.mp4 low_config_f.mp4
Parsing media file 1: high_config_f.mp4
Parsing media file 2: mid_config_f.mp4
Parsing media file 3: low_config_f.mp4
Splitting media file (video) high_config_f.mp4
Splitting media file (video) mid_config_f.mp4
Splitting media file (video) low_config_f.mp4
```

Ilustración 10. Comandos de preparación DASH de video

El resultado de este procedimiento será el directorio output con los ficheros de video y el documento stream.mdp asociado a la segmentación, visualizado en la Ilustración 11.

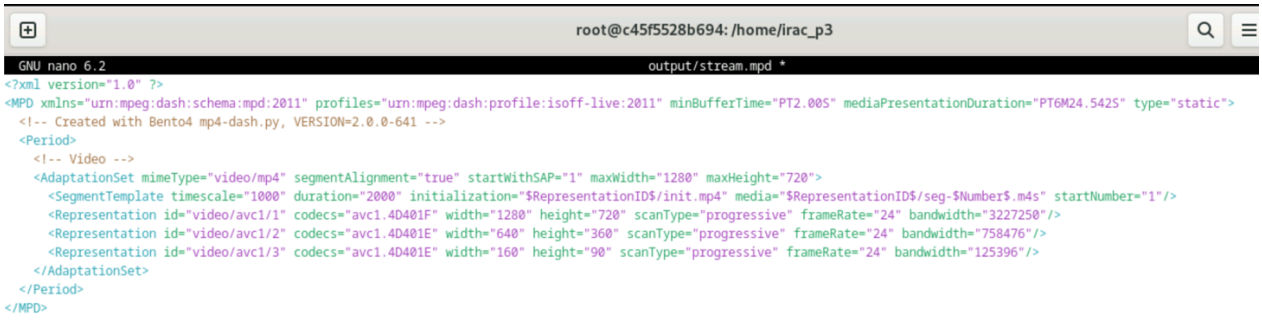


Ilustración 11. Documento MPD de la segmentación

Este documento se divide en varios campos los cuales nos dan la siguiente información:

- XML, indica que emplea la versión 1.0, necesario para que los procesadores interpreten correctamente el contenido
- MPD, este campo contiene las siguientes subetiquetas:
    - xmlns: el espacio de nombres del esquema MPEG-DASH (:urn:mpeg:dash:schema:mpd:2011)
    - profiles; indica el tipo de DASH utilizado: isoff-live.

    - minBufferTime, tiempo mínimo de búfer: 2 segundos.
    - mediaPresentationDuration, duración total del video: 6 minutos y 24.542 segundos
    - type: indica que el contenido es estático, es decir, pregrabado
- Period, agrupa los distintos conjuntos de representaciones disponibles durante un período de reproducción. En nuestro caso solo se emplea un periodo para todo el vídeo.
- AdaptationSet, representaciones que son alternativas entre sí. En nuestro caso:
    - mimeType, tipo de medio del que se trata el archivo: video/mp4.
    - segmentAlignment, indica que los segmentos sí están alineados temporalmente (: true).
    - startWithSAP="1", indica que comienza con un punto de acceso (para reproducción independiente).
    - maxWidth, resolución de ancho máxima disponible: 1280 px.
    - maxHeight, resolución de alto máxima disponible: 720 px.
- SegmentTemplate , define la estructura de los videos. Contiene las siguientes etiquetas:
    - timescale="1000", unidad de tiempo base (1000 uds por segundo, 1ms).
    - duration="2000", duración de cada segmento (2 segundos).
    - Initialization, archivo de inicialización para cada representación, en este caso es el fichero SRepresentationID$/init.mp4.
    - media, patrón de nombre para los segmentos de video, el cual es SRepresentationID$/seg-$Number$.m4s.
    - startNumber: primer segmento numerado, en este caso el primero.
- Representation, cada una define una calidad específica del video. En el fichero podemos observar que hay tres, asociadas a cada una de las calidades [alta / media / baja] :
    - id, identificador único del video.
    - códecs: avc1.4D401F indica que emplea H.264.
    - widht, anchura (en pixels) del video: [1280 / 640 / 160] px.
    - height, altura (en pixels) del video: [720 / 360 / 90] px.
    - scanType, tipo de escaneo: progresivo.
    - frameRate, tasa de cuadros por segundo: 24 fps.
    - bandwidth, ancho de banda estimado: [3227868 / 759262 / 125396] bps.

Finalmente, se ha implementado un cliente web para ofrecer un servicio de visualización del video ofrecido por envivo, que también presentará las métricas de la reproducción del archivo (bitrate, buffer y representación visualizada), generado por el fichero index.html.

Para esto, ejecutamos el servidor Python para acceder a la dirección localhost:8080, empleada en nuestro caso al levantar el Docker y poder visualizar el video.

Para la correcta visualización de los videos de diferentes calidades, generamos un archivo index.html en el que cargue el fichero con mayor calidad posible dentro de las capacidades del navegador. Es decir, el caudal de internet del navegador será bajado desde las opciones de desarrollador (como se ve en Ilustración 12) para observar como el video se observa sin cortes pasando por los tres archivos de distinto bitrate, optimizado para cada momento.

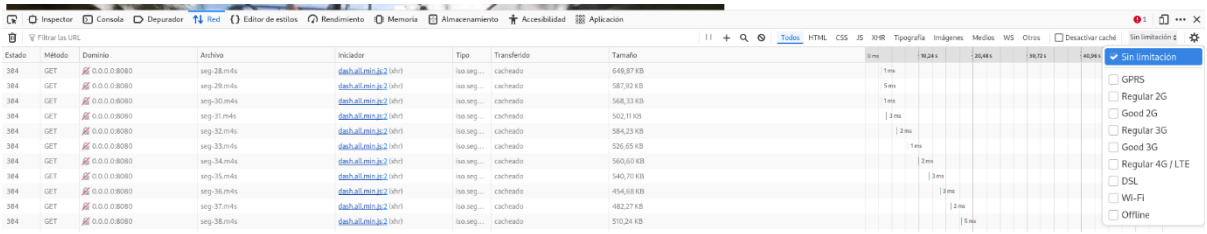


Ilustración 12. Limitaciones de red del navegador.

A continuación, vemos unas captura de la reproducción de cada uno de los videos generados, con la limitación de caudal necesaria :

Para la visualización del video con alta calidad, no es necesario imponer ninguna restricción de velocidad de red, como vemos en la Ilustración 13. Las estadísticas del video nos dan la información de que el video reproducido es avc1/1, que corresponde con el de mayor calidad, con un bitrate de 3228 kbps, el búfer máximo es de 31.292 segundos. Esto tiene sentido debido a que el bitrate real del video es de 2317.5 kbps y al ser mayor el bitrate percibido por el navegador, permite generar un búfer de una duración mayor.

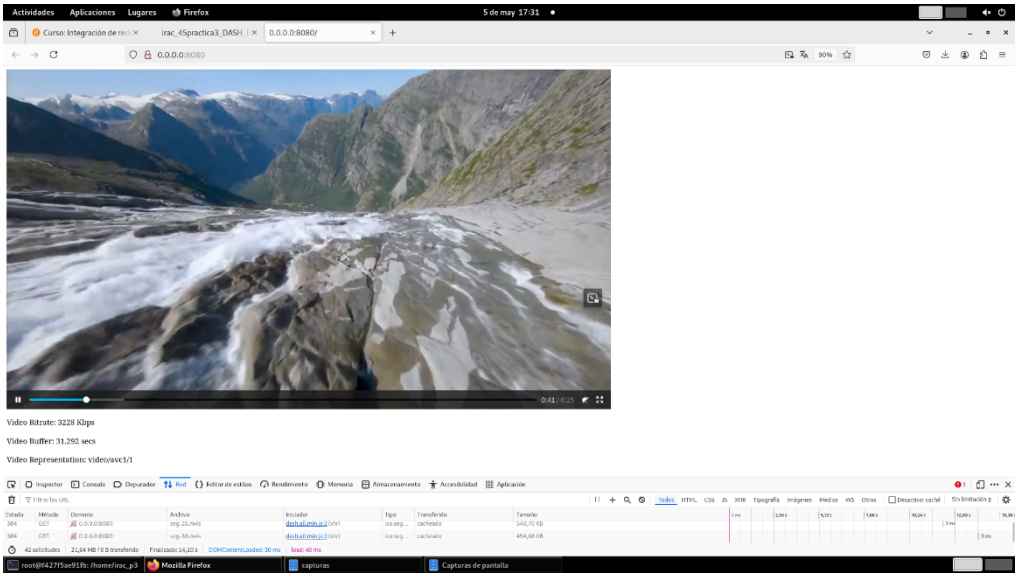


Ilustración 13. Visualización del video a alta calidad.

Por el contrario, para la visualización del video a calidad media (~600 kb/s), es necesario restringir la capacidad del navegador. En nuestro caso hemos optado por poner la limitación de “Regular 3G” de navegador Firefox, tras probar las posibilidades. Como vemos en la Ilustración 14,el video reproducido es avc1/2 (que correctamente es el de calidad media) su bitrate captado por el cliente web es de 739 kbps, mayor al real del video (579.2 kbps) lo que permite al búfer del video cargar un tiempo de reproducción mayor.

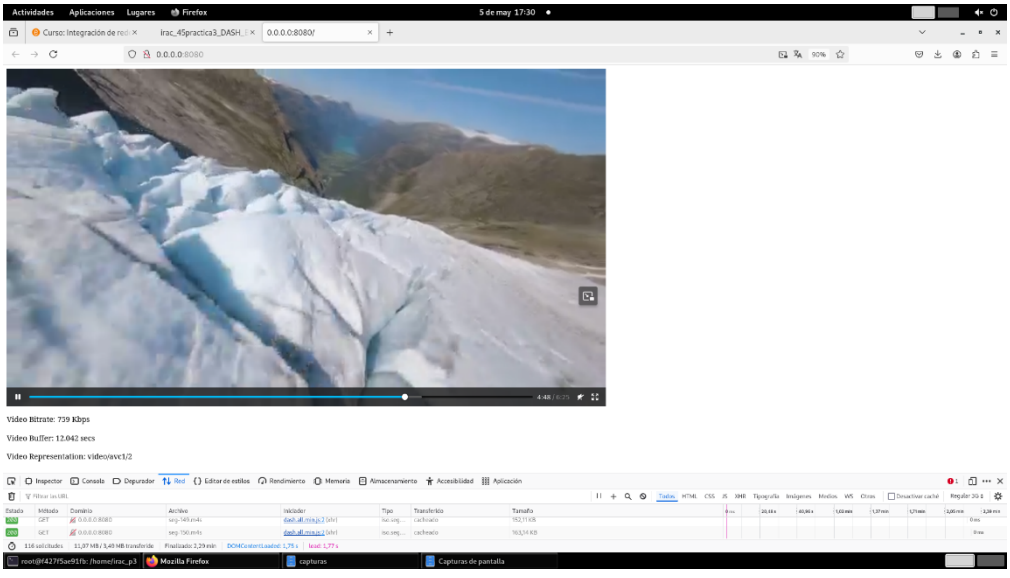


Ilustración 14. Visualización del video a calidad media.

Por último en la Ilustración 15, para visualizar el video generado con la peor calidad (~100 kbps), empleamos la limitación de “Good 2G”. Las métricas nos indican que se está reproduciendo el video avc1/3, que efectivamente es el de menor calidad de los

tres, a una velocidad de 125 kbps, siendo ligeramente mayor al bitrate real del video que es de 96.6 kbps. Esto permite cargar un buffer relativamente pequeño en comparación con los videos de otras calidades debido al margen mucho menor.

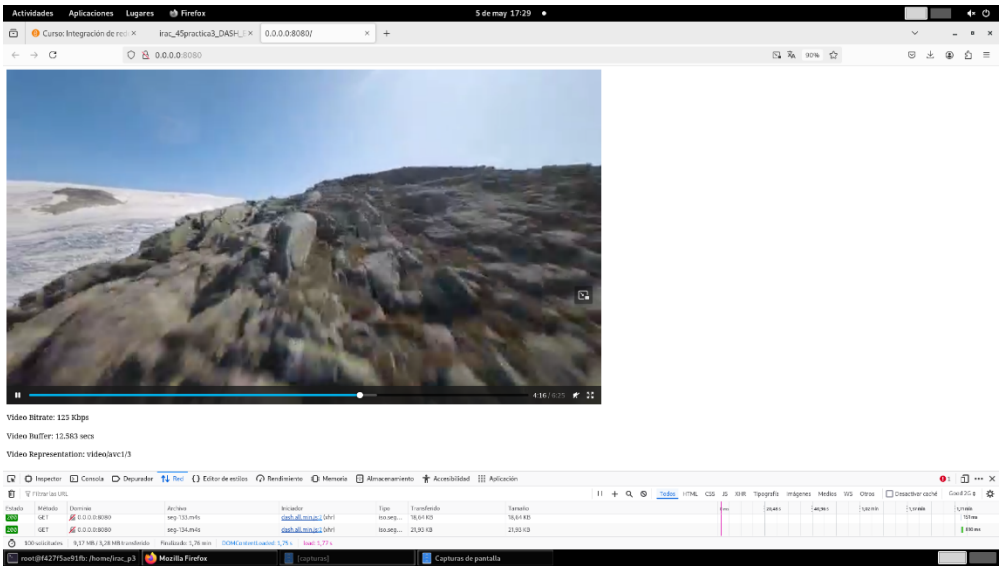


Ilustración 15. Visualización del video a baja calidad.

Para el entendimiento del protocolo Dash, nos fijamos en los mensajes de servidor en la pestaña de red como vemos en la Ilustración 16. Estos mensajes reflejan cómo el reproductor solicita dinámicamente los distintos segmentos del vídeo según las condiciones de red, cumpliendo el protocolo DASH.

Para tener esta situación en el servidor, es necesario desactivar las cachés del navegador, ya que si no el mensaje generado es un 304 (no modificado). Esto se debe a que no hay necesidad de que el servidor transfiera una representación del recurso de destino porque la solicitud indica que el cliente, que hizo la petición condicional, ya tiene una representación válida. De esta manera, el servidor está redirigiendo al cliente para que haga uso de esa representación almacenada como si fuera la carga útil de una respuesta 200 (OK). Esto podemos verlo en la Ilustración 16 donde el video se reproduce a calidad máxima debido a que no está limitada la capacidad de la red, sin embargo, todos los mensajes son 304 debido a que no están desactivadas las cachés.

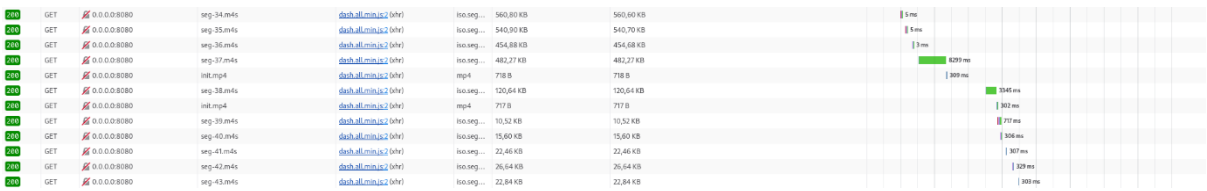


Ilustración 16. Visualización de mensajes GET - 200 OK dinámicos del servidor.

Además, los tiempos en verde (columna de duración) reflejan las latencias de carga, que también son útiles para el análisis de métricas.

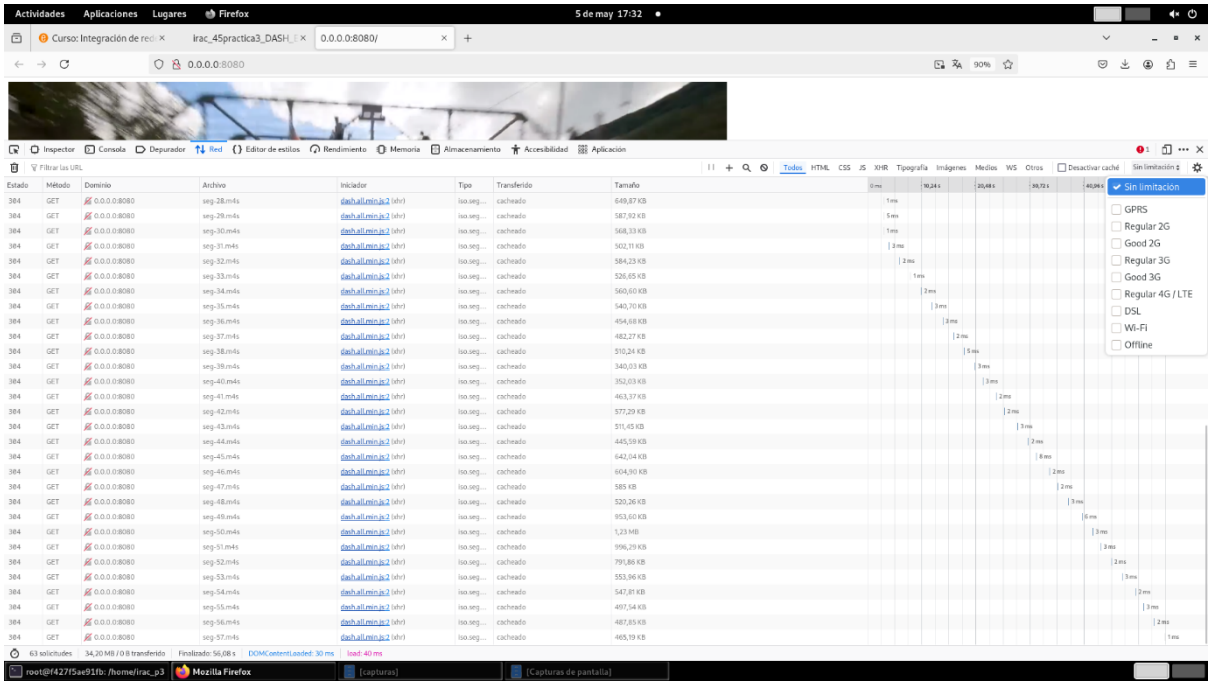


Ilustración 17. Visualización de mensajes de redirección debido a las cachés habilitadas.

Estos mensajes también pueden visualizarse en la consola donde hemos lanzado el servidor Python, como vemos en la Ilustración 18.

```
172.17.0.1 - - [05/May/2025 16:04:38] "GET /output/video/avc1/1/seg-29.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:39] "GET /output/video/avc1/1/seg-30.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:41] "GET /output/video/avc1/1/seg-31.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:44] "GET /output/video/avc1/1/seg-32.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:45] "GET /output/video/avc1/1/seg-33.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:48] "GET /output/video/avc1/1/seg-34.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:49] "GET /output/video/avc1/1/seg-35.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:51] "GET /output/video/avc1/1/seg-36.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:04:53] "GET /output/video/avc1/1/seg-37.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:02] "GET /output/video/avc1/2/init.mp4 HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:14] "GET /output/video/avc1/2/seg-38.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:17] "GET /output/video/avc1/3/init.mp4 HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:17] "GET /output/video/avc1/3/seg-39.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:18] "GET /output/video/avc1/3/seg-40.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:20] "GET /output/video/avc1/3/seg-41.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:22] "GET /output/video/avc1/3/seg-42.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:23] "GET /output/video/avc1/3/seg-43.m4s HTTP/1.1" 200 -
172.17.0.1 - - [05/May/2025 16:05:26] "GET /output/video/avc1/3/seg-44.m4s HTTP/1.1" 200 -
```

Ilustración 18. Mensajes del servidor web en la consola.

Estos mensajes muestran cómo el reproductor DASH (`dash.js`) está solicitando distintos archivos `.m4s` (segmentos de video) y `.mp4` (ficheros de inicialización) de manera dinámica. Pide los segmentos de videos por orden, hasta que la capacidad del navegador se ve limitada y es forzado a cambiar de calidad de video, por lo que pide el archivo init.mp4 de la configuración de calidad pertinente y tras ello ya puede

comenzar a solicitar los segmentos de video de esa calidad hasta que se vea exigido a cambiar de calidad de nuevo.

Se pueden observar en los archivos solicitados que sigue la configuración empleada en el archivo pmd: los segmentos de video duran 2 segundos aproximadamente, la forma de inicializar sigue el patrón especificado “avc1/1/init.mp4”, los segmentos se encuentran alineados temporalmente y comienzan por el 1.

<SegmentTemplate timescale="1000" duration="2000" initialization="SRepresentationID$/init.mp4 startNumber="1""

<Representation id="video/avc1/[1 / 2 / 3]"

< ... segmentAlignment="true" ...>

Parte 3: Gestión DRM con dash.js

En esta sección partimos de los videos fragmentados en la parte anterior.
En primer lugar se van a encriptar los tres videos segmentados, de las tres calidades, con mp4encrypt del framework Bento4.

```
root@f427f5ae91fb:/home/irac_p3# ./Bento4/bin/mp4encrypt --method MPEG-CENC --key 1:87237D20A19F58A740C05684E699B4AA:random --property 1:KID:A16E402B9056E371F36D348AA62BB749 --gl
obal-option mpeg-cenc.eme-pssh:true mid_config_f.mp4 mid_config_encrypt.mp4
root@f427f5ae91fb:/home/irac_p3# ./Bento4/bin/mp4encrypt --method MPEG-CENC --key 1:87237D20A19F58A740C05684E699B4AA:random --property 1:KID:A16E402B9056E371F36D348AA62BB749 --gl
obal-option mpeg-cenc.eme-pssh:true milow_config_f.mp4 low_config_encrypt.mp4
ERROR: cannot open input file (milow_config_f.mp4)
root@f427f5ae91fb:/home/irac_p3# ./Bento4/bin/mp4encrypt --method MPEG-CENC --key 1:87237D20A19F58A740C05684E699B4AA:random --property 1:KID:A16E402B9056E371F36D348AA62BB749 --gl
obal-option mpeg-cenc.eme-pssh:true low_config_f.mp4 low_config_encrypt.mp4
root@f427f5ae91fb:/home/irac_p3# ./Bento4/bin/mp4encrypt --method MPEG-CENC --key 1:87237D20A19F58A740C05684E699B4AA:random --property 1:KID:A16E402B9056E371F36D348AA62BB749 --gl
obal-option mpeg-cenc.eme-pssh:true high_config_f.mp4 high_config_encrypt.mp4
```

Ilustración 19. Encriptación de videos fragmentados

Con los comandos anteriores se han generado tres ficheros .mp4 con los videos fragmentados de cada una de las tres calidades encriptados con el método MPEG-CENC, definiendo una KEY y KID.

Para poder visualizar el video encriptado, es necesario procesar DRM en el cliente añadiendo lo relativo a la protección de la información, como se puede ver en fichero index.html de esta sección.

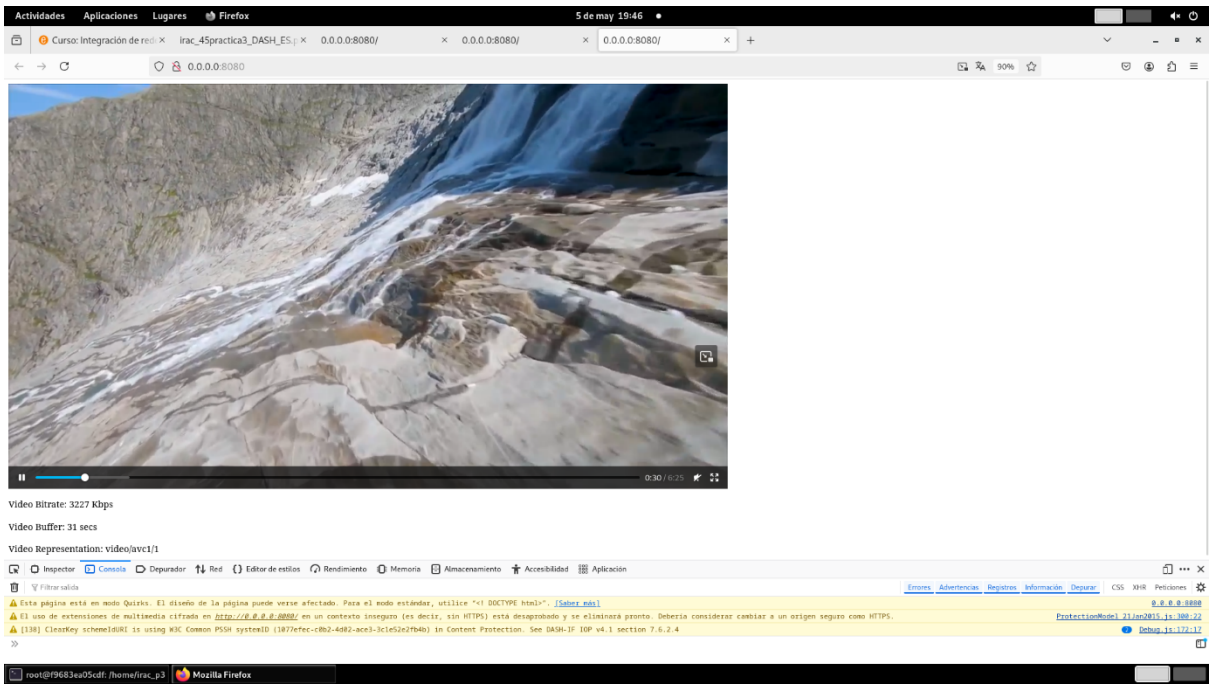


Ilustración 20. Comportamiento MPEG-DASH con DRM

En a la consola web de la Ilustración 20 se muestran dos mensajes relacionados con la encriptación:

“El uso de extensiones de multimedia cifrada en http://0.0.0.0:8080/ en un contexto inseguro (es decir, sin HTTPS) está desaprobado y se eliminará pronto.”
“ClearKey schemeIdURI is using WBC Common PSSH systemID (1077efec-c0b2-4d02-ace3-3c1e52e2fb4b) in Content Protection.”

El primer mensaje refleja que se está sirviendo contenido cifrado a través de HTTP y el segundo mensaje significa que se está utilizando ClearKey (un sistema de cifrado simple sin DRM propietario)

A continuación se realizó una alteración de las claves especificadas en protData en el fichero index.html, de modo que las claves definidas en el fichero ya no coincidan con las claves usadas en la encriptación.
Cuando no se tiene acceso a un vídeo encriptado el cliente no es capaz de reproducir el video, como se muestra en la Ilustración 21.

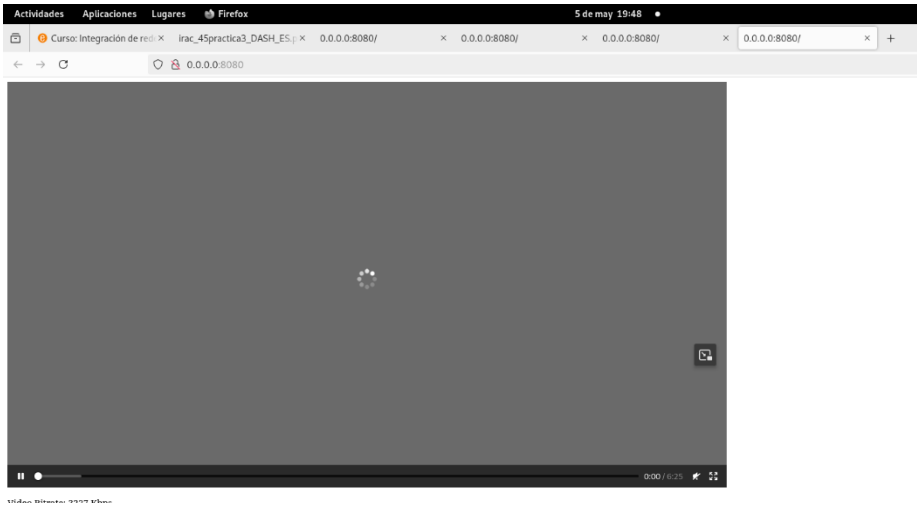


Ilustración 21. Visualización de contenido encriptado

En este caso, se obtienen los siguientes mensajes de error en la consola del navegador

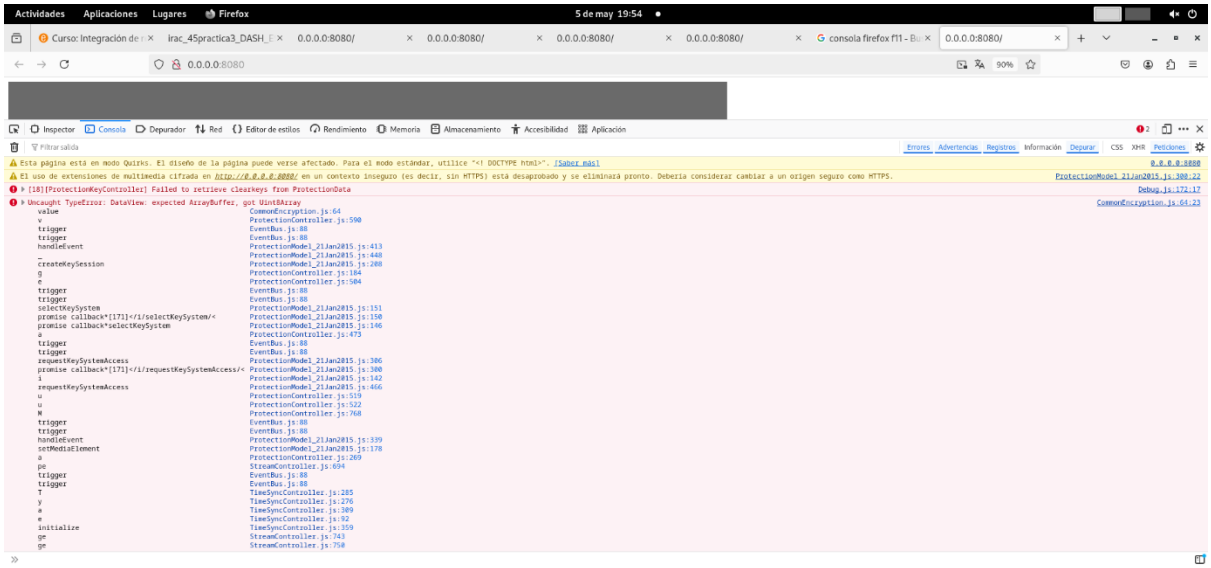


Ilustración 22. Mensajes de error de la consola por video sin acceso

El primer mensaje “[18] [ProtectionKeyController] Failed to retrieve clearkeys from ProtectionData” significa que El reproductor intenta acceder a las claves necesarias para descifrar el contenido encriptado, pero no encuentra las claves en el objeto ProtectionData

EL segundo error “Uncaught TypeError: DataView: expected ArrayBuffer, got Uint8Array” indica también un error en la conversión del contenido de la clave.

Parte 4: Visualización avanzada de métricas

En esta parte se va a mejorar de la visualización de las métricas asociadas al streaming de video mediante el protocolo DASH incorporando una gráfica dinámica que muestre a tiempo real la evolución del nivel del buffer y el bitrate del video.

La finalidad del grafico es proporcionar una representación visual del nivel de los indicadores clave de rendimiento del sistema al sufrir cambios de calidad.

Para su implementación se utilizó la librería Chart.js. A partir del reproductor dash.js, se extrajeron las mismas métricas de las secciones anteriores, Buffer Level (s) y Bitrate (bps). Estas métricas se capturaron cada 500 ms y se mostraron en formato de gráficas. Para poder visualizar claramente el comportamiento de las métricas, se fijaron en la gráfica los últimos 30 puntos recogidos.

En la siguiente imagen se muestra el resultado de la integración de la gráfica de visualización en el cliente, cuando se representa el video de mayor calidad,1.
Se representa en azul un bitrate constante de 3232Kbps, y en rosa el nivel del buffer que oscila en torno a los 30s

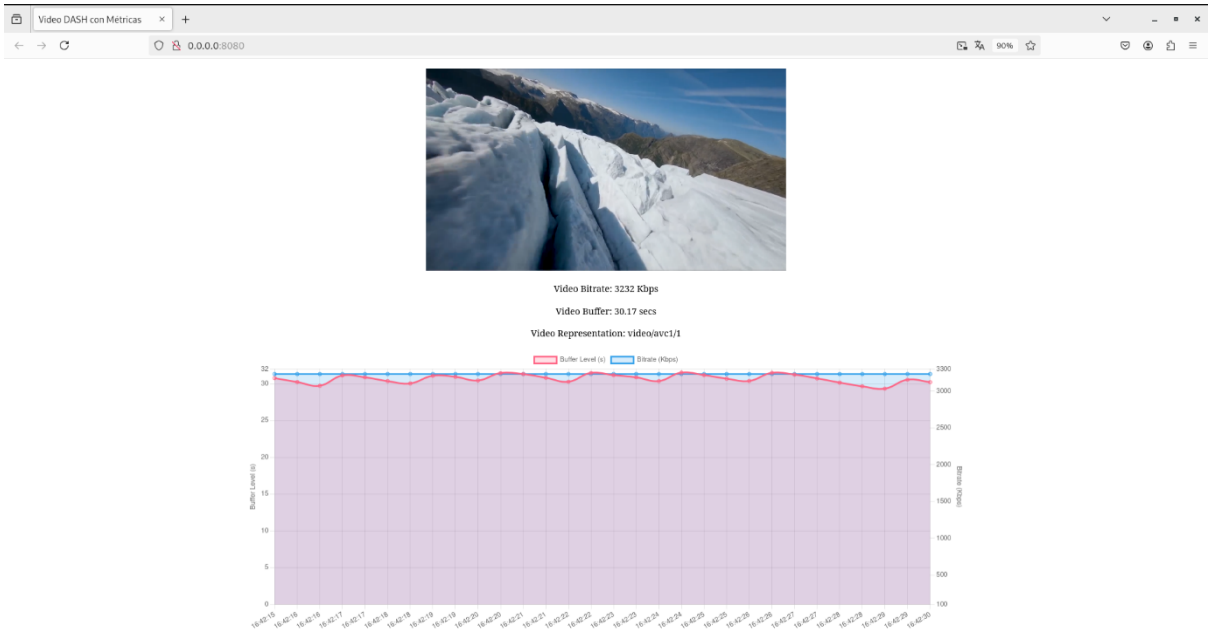


Ilustración 23. Métrica de streaming del video a calidad alta

A continuación, aplicando limitaciones a la red, visualizamos el salto de la representación 1 a la representación 2. El bitrate cae repentinamente en el momento del cambio a 760kbps. En cambio el buffer se vacía progresivamente como se ve en la Ilustración 24.

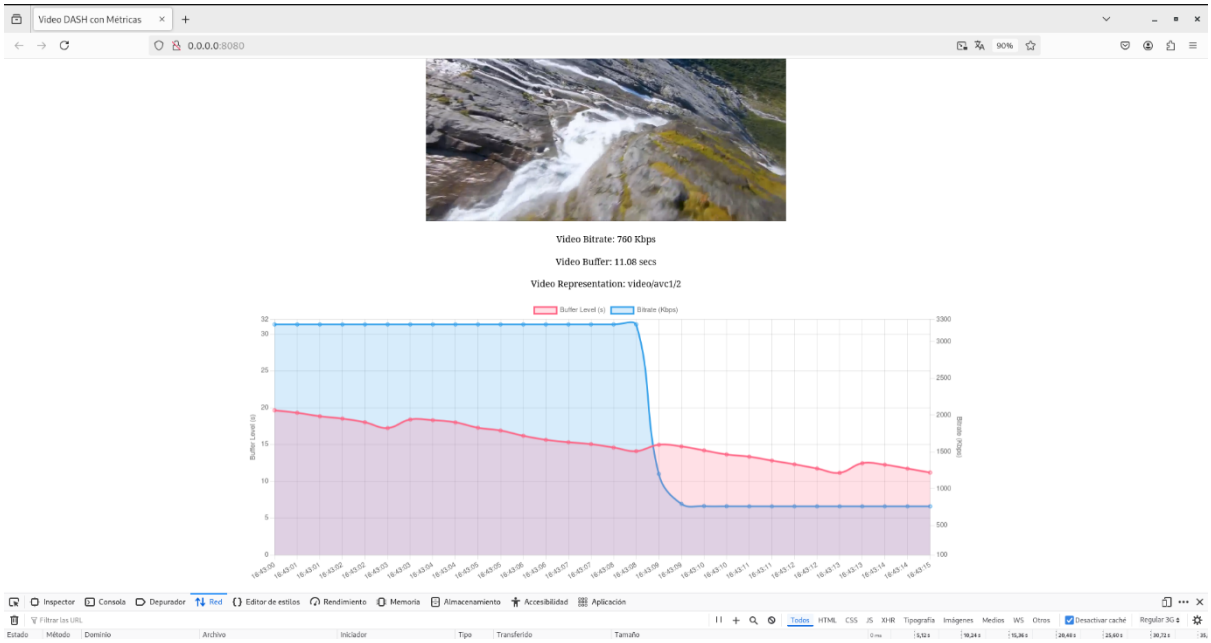


Ilustración 24. Métrica de streaming del cambio de video a calidad alta a media

Al mantener en el tiempo la calidad 2 del video el nivel del buffer se estabiliza en torno a 12s.

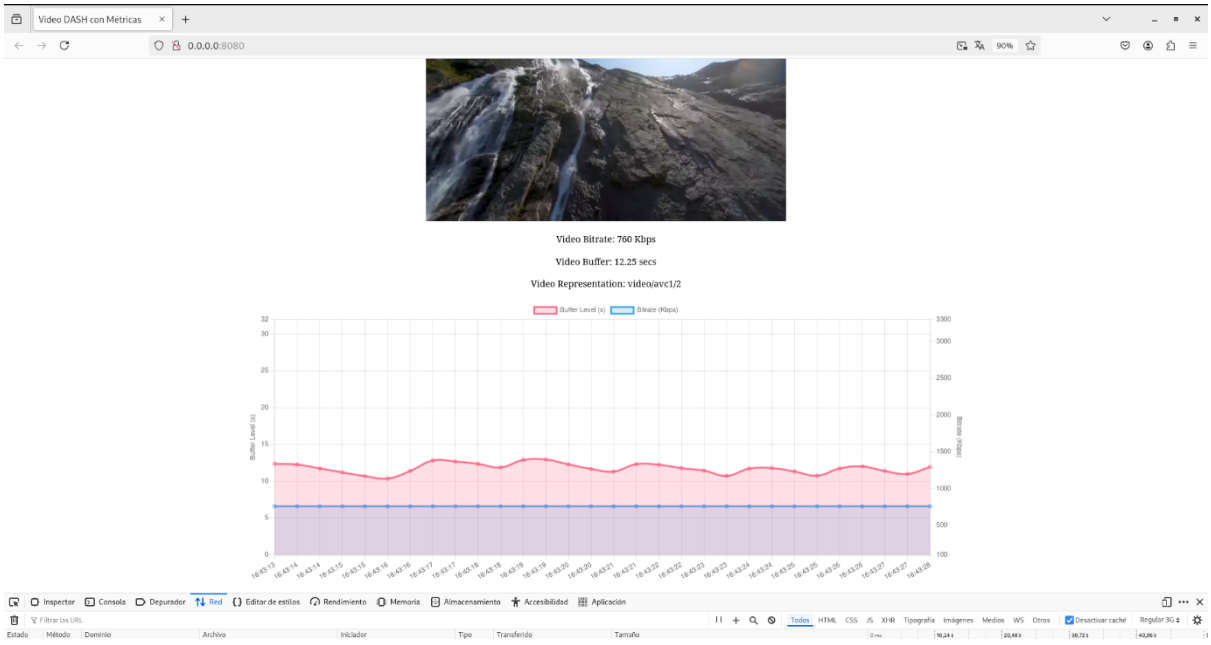


Ilustración 25. Métrica de streaming del video a calidad media

Del mismo modo que en el caso anterior, al aplicar una limitación a la red más restrictiva, observamos el cambio de la calidad del video a la calidad 3 (Ilustración 26).

El nivel del buffer se vacía hasta niveles muy bajos, antes de producirse el cambio de representación. En ese momento el bitrate desciende a 125kbps, lo que permite que el buffer comience a llenarse de nuevo (Ilustración 27).



Ilustración 26. Métrica de streaming del cambio de video a calidad media a baja



Ilustración 27. Métrica de streaming del cambio de video a calidad media a baja (llenado de buffer)

Al mantener en el tiempo el último nivel de calidad, el nivel del buffer vuelve a estabilizarse en torno a los 12s.



Ilustración 28. Métrica de streaming del video a calidad baja